CS336 作业2（系统）：系统与并行性

版本 1.0.4

2025年春季

1 作业概述

在本次作业中，你将获得一些实际经验，学习如何提升单GPU训练速度，并将训练扩展至多GPU环境。

你将要实现的内容。

1. 基准测试与性能分析工具集 2. Fla
sh Attention 2 Triton 内核 3. 分布
式数据并行训练 4. 优化器状态分片

代码的外观如下。所有作业代码以及本说明文档均可在GitHub上获取，地址是：

github.com/stanford-cs336/assignment2-systems

请 克隆仓库。如果有更新，我们会通知您，您可以 执行 git pull 来获取最新版本。

1. cs336-basics/：在本次作业中，你将对我们在作业1中构建的一些组件进行性能分析。此文件夹包含作业1的教员解决方案代码，因此你会找到一个cs336-basics/pyproject.toml以及一个 cs336-basics/cs336_basics/*模块。如果你希望使用自己实现的模型，可以修改基础目录中的 pyproject.toml文件，使其指向你的自定义包。

2. / ：cs336-系统的基本目录。我们创建了一个名为 cs336_systems 的空模块。请注意，这里没有任何代码，因此你可以从头开始自由实现任何功能。

3. tests/*.py：此文件包含您必须通过的所有测试。这些测试会调用在tests/adapters.py中定义的钩子。您将实现这些适配器，以将您的代码连接到测试中。编写更多测试和/或修改测试代码有助于调试您的代码，但您的实现仍需通过最初提供的测试套件。

4. README.md：此文件包含有关预期目录结构的更多详细信息，以及一些关于如何设置开发环境的基本说明。

ʜ 如何提交。 您将把以下文件提交至 Gradescope：

- 写作报告.pdf：请回答所有书面问题。请使用排版工具输入您的答案
- code.zip：包含你所编写的所有代码。

运行脚本 test_and_make_submission.sh，以生成 code.zip 文件。

在作业的第一部分，我们将探讨如何优化Transformer模型的性能，以实现对GPU的最高效利用。首先，我们会对模型进行性能剖析，了解其在前向和反向传播过程中分别消耗了哪些时间和内存；随后，我们将通过自定义的GPU内核来优化自注意力操作，使其比直接使用PyTorch实现的速度更快。在作业的后续部分，我们将进一步利用多块GPU来提升整体效率。

1.1 性能分析与基准测试

在实施任何优化之前，首先对程序进行性能分析非常有帮助，以便了解其资源消耗的热点（例如时间和内存）。否则，我们可能会误将模型中那些实际上并不占用大量时间和内存的环节作为优化目标，从而无法实现可衡量的端到端性能提升。

我们将实施三种性能评估路径：(a) 使用Python标准库进行简单、端到端的基准测试，以计时我们的前向和反向传播过程；(b) 利用NVIDIA Nsight Systems工具分析计算性能，深入了解时间在CPU和GPU上各操作间的分布情况；(c) 分析内存使用情况。

1.1.1 设置 - 导入您的基础Transformer模型

首先，让我们确保你能够成功加载上一节课的模型。在上一节课中，我们将模型设置在一个Python包内，以便日后能轻松导入。我们已在 ./cs336-basics 文件夹中添加了该模型的员工实现版本，并在 pyproject.toml 文件中指向了它。通过正常调用 uv run [command]， uv 将自动找到这个本地的cs336-basics 包。如果你希望使用自己实现的模型版本，可以修改 pyproject.toml 文件，将其指向你自己的包。

你可以通过以下方式测试是否能够成功导入你的模型：

```
~$ uv run python
使用 CPython 3.12.10
正在创建虚拟环境，路径为：/path/to/uv/env/dir
        构建 cs336-系统 @ 文件:///path/to/系统/目录
        已构建 cs336-基础 @ 文件路径：/path/to/basics/dir
已安装 85 个软件包，耗时 711 毫秒
Python 3.12.10 (主, 2025年4月9日, 04:03:51) [Clang 20.1.0 ] 在 Linux 上
...
>>> 导入 cs336_basics
>>>
```

现在应可使用来自作业1的相关模块（例如，对于 model.py，你可以通过 **import cs336_basics.model** 来导入）。

1.1.2 模型尺寸

在本次作业中，我们将对模型进行基准测试和性能剖析，以更深入地理解它们的表现。为了了解规模变化对模型性能的影响，我们将参考并使用以下几种模型配置。对于所有模型，我们将采用10,000的词汇量和4的批量大小，并尝试不同的上下文长度。此外，本次作业（以及后续作业）将需要大量结果以表格形式呈现。我们强烈建议您通过代码自动化生成报告所需的表格，因为使用LaTeX或Markdown格式化表格可能会非常繁琐。请参阅pandas.DataFrame.to_latex() 和 pandas.DataFrame.to_markdown() 或者编写您自己的函数，从首选的表格表示形式生成它们。

尺寸	d_model	d_ff	num_层	num头
小	768	3072	12	12
中	1024	4096	24	16
大型	1280	5120	36	20
xl	1600	6400	48	25
2.7B	2560	10240	32	32

表1：不同模型尺寸的规格

1.1.3 端到端基准测试

现在，我们将实现一个简单的性能评估脚本。由于我们将测试模型的多种变体（如调整精度、交换层等），因此通过命令行参数使脚本支持这些变体将非常有益，以便日后轻松运行。此外，我们强烈建议利用 Slurm 的 sbatch 或 submitit 工具，对基准测试中的超参数（如模型大小、上下文长度等）进行快速迭代的扫描实验。

首先，我们通过计时模型的前向和反向传播过程，对模型进行最简单的性能分析。由于我们将仅测量速度和内存占用，因此会使用随机初始化的权重和数据。

性能测量是一项微妙的工作——一些常见的陷阱可能导致我们无法准确衡量真正想要评估的内容。在对GPU代码进行基准测试时，一个需要注意的事项是：CUDA调用是异步的。当你调用一个CUDA内核，例如执行 torch.matmul时，该函数调用会立即将控制权返回给你的代码，而无需等待矩阵乘法操作完成。这样一来，CPU可以在GPU执行矩阵乘法的同时继续运行其他任务。但另一方面，这也意味着，如果我们简单地测量torch.matmul调用从开始到结束所花费的时间，并不能真实反映GPU实际执行矩阵乘法所需的时间。在PyTorch中，我们可以通过调用 torch.cuda.synchronize()来确保所有GPU内核均已执行完毕，从而获得更为精确的CUDA内核运行时间测量结果。基于此，让我们来编写基础的性能分析框架吧。

问题（基准测试脚本）：4分

(a) 编写一个脚本，用于对模型中的前向和反向传递执行基本的端到端基准测试。具体而言，您的脚本应支持以下功能：

- 给定超参数（例如，层数），初始化一个模型。
- 生成一批随机数据。
- 在开始计时前，先运行 个预热步骤；随后，测量执行 个步骤所需的时间（根据参数设置，可仅进行正向传播，或同时包含正向与反向传播）。计时操作可使用 Python 的 timeit 模块（例如，直接调用 timeit 函数，或采用 timeit.default_timer()，它能提供系统分辨率最高的时钟，因此比 time.time() 更适合作为基准测试的默认选择）。
- 每个步骤后调用 torch.cuda.synchronize()。

可交付成果：一个脚本，用于根据给定的超参数初始化一个基础Transformer模型，生成一批随机数据，并计时前向和反向传播过程。

(b) 对§1.1.2中描述的模型大小，分别计时前向和反向传播过程。使用5次预热循环逐步进行，并计算10次测量步骤中时间的平均值和标准偏差。前向传播需要多长时间？反向传播呢？你是否观察到各次测量之间存在较大波动，还是标准偏差较小？

可交付成果：用1-2句话说明你的计时结果。

(c) 基准测试的一个注意事项是，不要省略预热步骤。请重新执行你的分析，但这次不包括预热步骤。这会如何影响你的结果？你认为这是什么原因造成的？此外，尝试运行脚本时加入1或2个预热步骤。为什么即使这样，结果仍可能有所不同？

交付物：2-3句话的回复。

1.1.4 Nsight Systems 性能分析器

端到端的基准测试无法告诉我们模型在前向和反向传播过程中分别花费了哪些时间和内存，因此也无法揭示具体的优化机会。为了了解程序在各个组件（例如函数）上各耗时多少，我们可以使用性能分析器。执行性能分析器通过在函数开始和结束时插入检测代码来对目标代码进行插桩，从而能够提供详细的函数级执行统计信息，如调用次数、平均执行时间，以及该函数累计占用的总时间等。

标准的Python性能分析工具（例如，CProfile）无法对CUDA内核进行性能分析，因为这些内核是在GPU上异步执行的。幸运的是，NVIDIA随附了一个性能分析器，我们可以通过CLInsys来使用它——这个工具我们已为您安装完毕。在本次作业的这一部分，您将利用nsys来分析您的Transformer模型的运行时性能。使用nsys非常简单：只需在之前章节的Python脚本前加上nsys profile即可运行。例如，您可以对脚本benchmark.py进行性能分析，并将输出结果写入文件result.nsys.rep，具体命令如下：

```
~$ uv run nsys profile -o result python benchmark.py
```

然后，您可以在本地机器上使用 NVIDIA Nsight Systems 桌面应用程序查看该性能分析报告。在报告的 CUDA API行中选择某个特定的 CUDA API 调用（位于 CPU 上），将高亮显示 CUDA HW行中所有对应的 GPU 内核执行情况。

我们鼓励您尝试使用 nsys profile 的各种命令行选项，以深入了解其功能。特别值得一提的是，通过添加 --python-backtrace=cuda 参数，您可以获取每个CUDA API调用的Python回溯信息，尽管这可能会带来一定的性能开销。此外，您还可以在代码中插入NVTX范围标记，这些标记会在配置文件的 NVTX 行中显示为区块，从而捕获所有CUDA API调用及其关联的内核执行情况。尤其需要注意的是，您应利用NVTX范围标记来 **忽略基准测试脚本中的预热步骤** (方法是在配置文件的 NVTX 行上应用过滤器）。同时，您还可以明确哪些内核负责模型的前向传播和反向传播过程；甚至可以通过如下方式为您的实现添加注释，进而区分自注意力层中不同部分所对应的内核：

```
...
导入 torch.cuda.nvtx 作为 nvtx

@nvtx.range("缩放点积注意力")
def带注释的缩放点积注意力机制(
    ... # Q, K, V, mask
)
    ...
    with nvtx.range("计算注意力分数"):
        ... # 计算 Q 和 K 之间的注意力分数

    与 nvtx.范围（"计算Softmax"）
        ... # 计算注意力分数的Softmax

    与 nvtx.范围（"最终矩阵乘法"）
        ... # 计算输出投影
```

```
    返回 ...
```

您可以通过以下方式，在基准测试脚本中用带注释的版本替换您的原始实现：

```
cs336_基础.模型.缩放点积注意力 = 注释化缩放点积注意力
```

最后，你可以使用 --pytorch 命令行选项配合 nsys，自动为 PyTorch C++ API 的调用添加 NVTX 范围标记。

问题（nsys_profile）：5分

使用 nsys 对表1中描述的每种模型尺寸，以及上下文长度为128、256、512和1024的情况，分别对前向传播、反向传播和优化器步骤进行性能剖析；请注意，对于较大规模的模型，部分上下文长度可能会导致内存不足，此时请在报告中注明。

(a) 您的前向传播总共花费了多少时间？这是否与我们之前使用 Python 标准库测量的结果一致？

可交付成果：用1到2句话作答。

(b) 在前向传播过程中，哪个CUDA内核占用的GPU时间最多？具体是多少？
在模型的单次前向传播过程中，此内核被调用了多少次？当你同时执行前向和反向传播时，是否是同一个内核占据了最多的运行时间？（提示：查看“Stats Systems View”下的“CUDA GPU 内核摘要”，并利用 NVTX 范围进行筛选，以确定模型中哪些部分负责生成哪些内核。）

交付物：1-2句话的回复。

(c) 尽管绝大多数的浮点运算发生在矩阵乘法中，但你会发现，其他几种内核仍然占据了整体运行时间中不可忽视的部分。除了矩阵乘法之外，你还观察到哪些内核在前向传播过程中占用了相当多的CUDA运行时间？

交付物：1-2句话的回复。

(d) 运行一个完整训练步骤的性能分析，使用您实现的AdamW算法（即前向计算部分通过计算损失并执行反向传播，最后再进行优化器步骤，这与训练过程中的操作完全一致。那么，与仅执行前向传播的推理阶段相比，矩阵乘法所花费的时间占比会发生怎样的变化？其他内核呢？
交付物：1-2句话的回复。

(e) 比较模型前向传播过程中，softmax运算与自注意力层内矩阵乘法运算的运行时间。这种运行时间差异与浮点运算次数（FLOPs）差异相比如何？

交付物：1-2句话的回复。

1.1.5 混合精度

到目前为止，我们在作业中一直采用FP32精度——模型的所有参数和激活函数都使用了torch.float32数据类型。然而，现代NVIDIA GPU配备了专门的GPU核心（Tensor Core），能够以更低的精度加速矩阵乘法运算。例如，NVIDIA A100的技术规格表显示，其FP32的最大吞吐量为19.5 TFLOP/秒，而使用FP16（半精度浮点数）或BF16（脑浮点数）时，最大吞吐量则显著提升至312 TFLOP/秒。因此，采用低精度数据类型有助于加快训练和推理的速度。

然而，若简单地将模型转换为较低精度的格式，可能会导致模型精度下降。例如，在实际应用中，许多梯度值往往小到无法用FP16表示，因此在使用FP16精度进行训练时会直接变为零。为应对这一问题，通常可在FP16训练中采用损失缩放技术——即简单地将损失乘以一个缩放因子，从而放大梯度幅度，避免其被舍入为零。此外，FP16的动态范围低于FP32，这可能导致溢出现象，进而使损失变为NaN。相比之下，完全使用bfloat16进行训练通常更为稳定（因为BF16与FP32具有相同的动态范围），但即便如此，与FP32相比，最终的模型性能仍可能受到影响。

为了充分利用低精度数据类型的加速优势，通常采用混合精度训练。在PyTorch中，这通过torch.autocast上下文管理器来实现。在这种情况下，某些操作（例如矩阵乘法）会以低精度数据类型执行，而其他需要FP32完整动态范围的操作（例如累加和约简）则保持原样。例如，以下代码将自动识别前向传播过程中哪些操作应以低精度执行，并将这些操作转换为指定的数据类型：

```
模型：torch.nn.Module = ... # 例如你的Transformer模型 数据
类型：torch.数据类型 = ... # 例如，torch.float16
x：torch.Tensor = ... # 输入数据

与 火炬.autocast(device="cuda",dtype=dtype):
    y = 模型(x)
```

正如上文所暗示的，即使待累积的张量已进行降级类型转换，通常仍建议以更高的精度保存累积结果。以下练习将帮助你更好地理解其中的原因。

问题 (mixed_precision_accusion): 1分

运行以下代码并对 (的准确性) 结果进行commment。

```
s = torch.tensor(0,dtype=torch.float32)
for
    s += torch.tensor(0.01,dtype=torch.float32)
打印 (s)

对于i,s = torch.tensor(0,dtype = torch.float16)
    i in range(1000):
    s += torch.tensor(0.01,dtype=torch.float16)
打印 (s)

    s += torch.tensor(0.01,dtype=torch.float16)
打印 (s)

    x = torch.tensor(0.01,dtype=torch.float16)
    s += x.type(torch.float32)
打印 (s)
```

可交付成果: 2–3句回答。

现在，我们将首先将混合精度应用于一个玩具模型以帮助理解，然后再应用到我们的基准测试脚本中。

问题（混合精度基准测试）：2分

(a) 考虑以下模型：

```
1 类 ToyModel(nn.Module):
2         def __init__(self, in_features: int, out_features: int):
3             超级().__init__()
4             self.fc1 = nn.线性（输入特征数， 10，偏置=False）
5             self.ln = nn.LayerNorm(10)
6             self.fc2 = nn.Linear(10, out_features, bias=False)
7             self.ReLU = nn.ReLU()
8
9         def forward(self, x):
10            x = self.relu(self.fc1(x))
11            x =  self.ln(x)
12            x =  self.fc2(x)
13            返回 x
```

假设我们正在GPU上训练模型，且模型参数最初以FP32格式存储。现在，我们希望使用自动混合精度并启用FP16。那么，以下数据类型的分别是：

- 自动混合精度上下文中的模型参数，
- 第一个前馈层的输出（ToyModel.fc1），
- 层归一化层的输出（ToyModel.ln），
- 模型预测的 logits，
- 损失，
- 以及模型的梯度？

可交付成果：列出的各组件的数据类型^ 五。

(b) 你应该注意到，FP16混合精度自动类型转换会对待层归一化层与前馈层不同。层归一化中哪些部分对混合精度敏感？如果我们改用BF16而非FP16，是否仍需对层归一化进行特殊处理？为什么？

交付物：2-3句的简要回答。

(c) 修改你的基准测试脚本，使其能够选择性地使用BF16混合精度运行模型。针对§1.1.2中描述的每种语言模型规模，分别测量启用和禁用混合精度时的前向与反向传播耗时。比较全精度与混合精度下的实验结果，并根据模型规模的变化趋势进行分析和讨论。你可能会发现 nullcontext 无操作上下文管理器对你有所帮助。

可交付成果：用2到3句话回答，包括你的计时和评论 初级。

1.1.6 内存剖析

到目前为止，我们一直在关注计算性能。接下来，我们将把注意力转向内存，这是语言模型训练和推理中另一个关键资源。PyTorch还内置了一个功能强大的内存分析器，能够实时跟踪内存分配情况。

要使用内存分析器，您可以按如下方式修改您的基准测试脚本：

```
...# 在你的基准测试脚本中的预热阶段

  # 开始记录内存历史。
火炬.cuda.内存._记录内存历史（最大条目数=1000000）

...# 您希望在基准测试脚本中剖析的内容

  # 将一个 pickle 文件保存，以便 PyTorch 的在线工具加载。
火炬.CUDA.内存._dump_snapshot("memory_snapshot.pickle")

  # 停止记录历史。
火炬.CUDA.内存._记录内存历史（启用=无）
```

这将输出一个文件memory_snapshot.pickle，你可以将其加载到以下在线工具中：https://pytorch.org/memory_viz 。该工具可让你查看整体的内存使用时间线，以及每次单独的内存分配情况，包括其大小和指向分配源代码的堆栈跟踪信息。要使用此工具，你应先在网页浏览器中打开上述链接，然后将你的Pickle文件拖放到页面上。

现在，您将使用 PyTorch 性能分析器来分析模型的内存使用情况。

问题（内存分析）：4分

对表1中27亿参数模型的前向传播、反向传播及优化器步骤进行性能剖析，实验设置的上下文长度分别为128、256和512。

(a) 在你的性能分析脚本中添加一个选项，以便通过内存分析器运行模型。建议复用你之前的一些基础设施（例如，用于启用混合精度、加载特定模型尺寸等）。然后，运行该脚本，获取27亿参数模型在仅执行推理（仅前向传播）或完成完整训练步骤时的内存使用情况概览。你的内存时间线看起来如何？能否根据观察到的峰值判断当前正在执行哪个阶段？

交付物：来自 memory_viz 工具的、一个27亿参数模型的“活跃内存时间线”图像两张，一张展示前向传播过程，另一张展示完整训练步骤（包括前向和反向传播，随后执行优化器步骤）；以及一段2至3句的简要说明。

(b) 在执行前向传播时，每种上下文长度的峰值内存使用量是多少？至于执行完整训练步骤时？

可交付成果：一张表格，每种上下文长度包含两个数字。

(c) 找出在使用混合精度时，2.7B模型的峰值内存占用情况，包括前向传播和完整优化器步骤。混合精度是否显著影响内存使用？

交付物：一段2至3句话的回复。

(d) 考虑 2.7B 参数量的模型。在我们的参考超参数设置下，一个张量的大小是多少？在Transformer残差流中，以单精度表示的激活值占用多少MB内存？请将字节数除以1024，再除以1024得到结果。

可交付成果：用1-2句话回答，并附上推导过程。

(e) 现在仔细查看来自 pytorch.org/memory_viz 的“活跃内存时间线”，它展示了执行前向传播时，一个 27亿参数 模型的内存快照。当你降低“详细程度”级别时，该工具会根据相应级别隐藏最小的内存分配（例如，将“详细程度”设为10%时，仅显示

（占比前10%的最大分配）。所显示的最大分配的大小是多少？通过查看堆栈跟踪，你能确定这些分配的来源吗？

交付物：1-2句的简要回答。

1.2 使用FlashAttention-2优化注意力机制

1.2.1 PyTorch注意力机制的基准测试

你的性能分析很可能表明，你的注意力层在内存和计算方面都存在优化空间。从高层次来看，注意力操作包括一次矩阵乘法，接着是Softmax函数，最后再进行一次矩阵乘法：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\text{mask}\left(\frac{Q^\top K}{\sqrt{d_k}}\right)\right)V \tag{1}$$

朴素的注意力实现需要为每个批次/头元素保存形状为 seq_len × seq_len 的注意力分数矩阵，而随着序列长度的增加，这些矩阵可能会变得非常庞大，从而导致任何处理长输入或输出任务时出现内存不足的错误。为此，我们将按照FlashAttention-2论文实现一个注意力核函数，该函数通过分块方式计算注意力，并完全避免显式地生成 seq_len × seq_len 的注意力分数矩阵，从而支持扩展到更长的序列长度。

问题（PyTorch注意力机制）：2分

(a) 基准　请关注不同规模下的实现细节。编写一个脚本　那将：

(a) 将批大小固定为8，并且不使用多头注意力机制（即移除头维度）。(b) 对于头嵌入维度 dmodel，遍历 [16, 32, 64, 128] 的笛卡尔积；对于序列长度 [256, 1024, 4096, 8192, 16384] 进行迭代。

(c) 为适当大小创建随机输入 Q、K、V。

(d) 对输入执行100次前向注意力传播。

(e）在反向传播开始前，测量当前已使用的内存大小，并记录100次反向传播所需的时间。

(f) 请务必进行热身，并在每次前向或反向传播后调用 torch.cuda.synchronize()。

请报告这些配置下的运行时间（或内存不足错误）。在多大规模时会出现内存不足的错误？针对其中一个最小但确实导致内存耗尽的配置，计算一下注意力机制所占用的内存（可参考作业1中给出的Transformer内存使用公式）。此外，随着序列长度的变化，反向传播节省的内存将如何变化？你又将采取哪些措施来彻底消除这一内存开销？

交付物：一张包含您记录的时间、内存使用情况的计算过程，以及1到2段的简要说明。

1.3 JIT编译注意力机制的基准测试

自2.0版本起，PyTorch还内置了一个功能强大的即时编译器，可自动尝试对PyTorch函数应用多种优化措施：详情请参阅 https://pytorch.org/tutorials/intermediate/torch_compile_tutorial.html以获取入门介绍。特别是，该编译器会通过动态分析你的计算图，自动生成融合的Triton内核。使用PyTorch编译器的接口非常简单。例如，若我们希望将其应用于模型中的单个层，只需如下操作即可：

```
1 层 = SomePyTorchModule(...)
2 编译层 = torch.compile(层)
```

现在，compiled_layer在功能上完全等同于layer（例如，具备前向和反向传播能力）。我们还可以使用torch.compile(model)来编译整个PyTorch模型，甚至可以直接编译调用PyTorch操作的Python函数。

问题（torch_compile）：2分

(a) 扩展你的注意力基准测试脚本，加入你使用PyTorch实现的注意力机制的编译版本，并将其性能与未编译版本进行对比，两者均采用与上述pytorch_attention问题相同的配置。

可交付成果：一张表格，用于比较您已编译的注意力模块与上述 pytorch_attention 问题中未编译版本的前向和后向传递时间。

(b) 现在，在你的端到端基准测试脚本中编译整个Transformer模型。那么，如何前向传播的性能有变化吗？那么，结合了前向与反向传播以及优化器步骤的情况又如何呢？

可交付成果：一张对比您的原始Transformer模型与编译后Transformer模型的表格。

鉴于我们所观察到的序列长度相关的扩展行为，我们需要大幅改进以有效处理长序列。即便使用 torch.compile，当前实现仍面临长序列时极差的内存访问模式问题。为此，我们将基于Triton编写FlashAttention-2的实现，从而更灵活地控制内存访问方式，并精确决定何时计算何物。

1.3.1 示例 - 加权求和

为了介绍您需要了解的关于Triton及其与PyTorch互操作性的内容，我们将通过一个“加权求和”操作的示例内核来逐步讲解。如需更多帮助以快速上手Triton，可参阅Triton的教程。需要注意的是，这些教程并未采用我们接下来将详细介绍的全新、便捷的块指针抽象机制。

给定输入矩阵 X，我们将按列将其元素与权重向量 w相乘，并对每一行求和，从而得到矩阵与向量的乘积 X 和 w。首先，我们来详细推导这一操作的前向传播过程，随后再编写用于反向传播的Triton内核。

前向传播 我们的核函数的前向传播步骤就是以下广播后的内积运算。

```
1 def加权和(x, weight):
2     # 假设 x 的形状为 [...] × D，而权重的形状为 [D] 返回 (权重 * x).沿轴=-1求和
3
```

在编写我们的Triton内核时，我们将让每个程序实例（可能并行运行）计算一个 x矩阵中行的 tile的加权和，并将相应的标量输出写入输出张量。在Triton中，一个程序实例是一组线程块，这些线程块中的所有线程都执行相同的程序，并且这些线程块 可以在GPU上并行运行。此外，我们不再直接以张量作为参数，而是传入指向它们第一个元素的指针，以及每个张量的步长，用于指示如何沿各个轴移动。

我们可以利用步长加载与当前实例中正在求和的 x行块对应的张量，并通过程序ID来划分工作（即，实例 i将处理 i号的 x行块）。在这个简单案例中，Triton与PyTorch前向传播的主要区别在于，我们需要手动进行指针算术运算，并显式地执行加载/存储操作。接下来，我们将使用带有的块指针抽象方法。

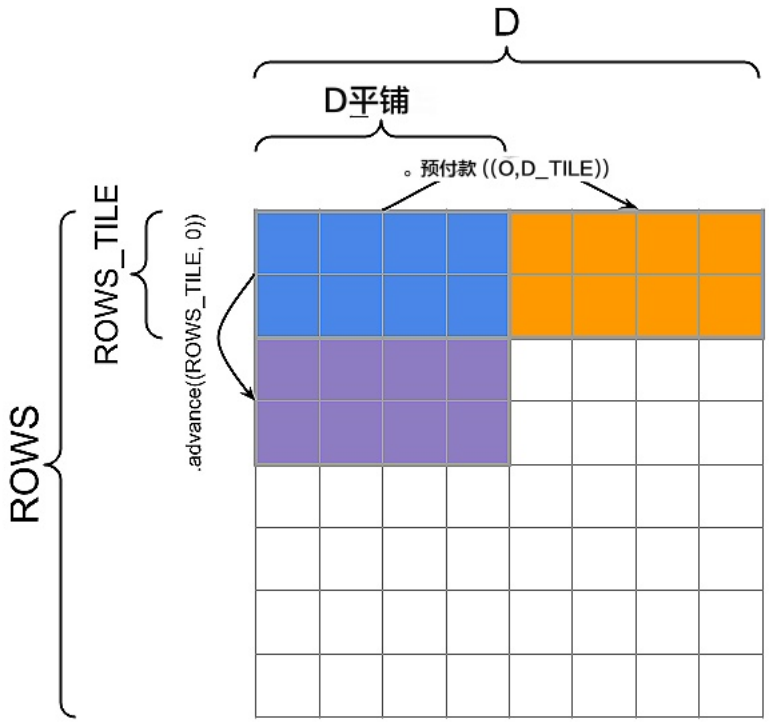

图1：加权求和核示例（第1.3.1节）中的分块与块指针推进过程。

tl.创建块指针，以极大简化指针算术，尽管这意味着我们需要进行一些设置来准备块指针。

请参考图1，了解铺 tile 的示意图以及如何推进块指针。上述的加权求和函数如下所示：

```
import 特里同
导入 特里同语言 作为 tl

@triton.jit
def加权和前向传播(
        x_ptr, weight_ptr, # 输入指针
        输出指针， # 输出指针
        x_stride_row, x_stride_dim, # 步长指示了在张量每个轴上如何移动一个元素
        权重步幅维度， # 可能是1
        输出步长行， # 可能为 1
        行，D，
        ROWS_TILE_SIZE：tl.常量，D_TILE_SIZE：tl.常量， # 瓦片形状必须在编译时已知
):
        # 每个实例将计算 x 的一行瓷砖的加权和。
        # `tl.program_id` 为我们提供了一种方法，以检查当前正在运行的线程块
        行块索引 = tl.程序ID(0)

        # 块指针为我们提供了一种从非连续内存区域中进行选择的方法
        # 并移动我们的选区。
        # 块指针必须知道：
        # - 指向张量第一个元素的指针
        # - 张量的整体形状，用于处理越界访问
```

```
    # - 用于正确安排内存布局的各维度步长 # - 起始块的 ND 坐标，即 “偏移量”

    # - 每次加载/存储时使用的块形状 # - 内存中维度从主到次的排列顺序（即 `np.argsort(strides)`），用于优化，尤其在 H100 上效果显著


    x_block_ptr = tl.make_block_ptr(
        x指针，
        形状=(行数, D,),
        步幅=(x_row_stride, x_stride_dim), 偏移量=(行平铺索引 * 行平铺大小, 0), 块形状=(行平铺大小, 列平铺大小), 顺序=(列优先, 0),


    )

    weight_block_ptr = tl.make_block_ptr(
        权重指针，
        形状=(D,),
        步长=(权重步长维度),
        偏移量=(0,),
        块形状=(D_TILE_SIZE,),
        排序=(0,),
    )

    输出块指针 = tl.创建块指针(
        输出指针，
        形状=(行数,),
        步长=(输出步长行数),
        偏移量=(行块索引 * 行块大小,), 块形状=(行块大小,),

        排序=(0,),
    )

    # 初始化一个缓冲区以供写入
    输出 = tl.zeros((ROWS_TILE_SIZE,), dtype=tl.float32)

    for i in range(tl.cdiv(D, D_TILE_SIZE)):
        # 加载当前块指针
        # 由于 ROWS_TILE_SIZE 可能无法整除 ROWS，而 D_TILE_SIZE 可能无法整除 D，# 因此我们需要对两个维度都进行边界检查
        行 = tl.load(x_block_ptr, boundary_check=(0, 1), padding_option="zero") # (ROWS_TILE_SIZE, D_TILE_SIZE) weigh
        t = tl.load(weight_block_ptr, boundary_check=(0,), padding_option="zero") # (D_TILE_SIZE,)

        # 计算该行的加权和。
        输出 += tl.sum(行 * 权重[索引None], 轴=1)

        # 将指针移动到下一个方块。
        # 这些是（行、列）坐标增量 x_block_ptr = x_block_ptr.advance((0, D_TILE_SIZE)) # 沿最后一个维度移动 D_TILE_SIZE 距离 weight_block_ptr = weight_block_ptr.前进((D_TILE_SIZE,)) # 每次移动 D_TILE_SIZE


    # 将输出写入输出块指针（每行一个标量）。
    # 由于 ROWS_TILE_SIZE 可能无法整除 ROWS，我们需要进行边界检查 tl.存储(output_block_ptr, 输出, 边界检查=(0,))

```

现在，让我们将这个内核封装进一个PyTorch Autograd函数中，以便与PyTorch无缝交互（即，接收张量作为输入，输出一个张量，并在反向传播过程中也能与Autograd引擎协同工作）：

```
类 加权求和函数(torch.autograd.函数）：
@staticmethod
    def 前向传播(ctx, x, weight):
        # 缓存用于反向传播的x和权重，以便在我们
        # 仅接收相对于输出张量的梯度，并且
```

```
        # 需要计算关于 x 和权重的梯度。
        D，output_dims = × 形状[-1]，x. 形状[:-1]

        # 将输入张量重塑为2D形状
        输入形状 = x.形状
        x = 重新排列(x，"... d -> (...) d")

        ctx.保存反向传播所需信息(x, weight)

        断言 len(权重.形状) == 1 且 权重.形状[0] == D， "维度不匹配" 断言 x.是否为CUDA张量，以及 权重.是否为CUDA张量，"预期CUDA张量"
        断言 x.是连续的，“我们的指针算术将假定 x 是连续存储的”

        ctx.D_TILE_SIZE = 特里顿.下一个2的幂(D) // 16 # 大约16次循环遍历嵌入维度 上下文.ROWS_TILE_SIZE = 16 # 每个线程一次处理16个批次元素
        ctx.input_shape = 输入形状

        # 需要初始化空的结果张量。请注意，这些元素并不一定都是0！ y = torch.empty(output_dims, device=x.device)


        # 在我们的1D网格中启动包含n个实例的内核。
        n_rows = y.numel()
        加权和前向传播[(cdiv(行数, 上下文.行块大小),)](
            x，权重，
            y，
            x.步长(0)，x.步长(1)，
            权重.步长(0)，
            y.步幅(0)，
            行=n_行，D=D，
        ROWS_TILE_SIZE=ctx.ROWS_TILE_SIZE, D_TILE_SIZE=ctx.D_TILE_SIZE，)


        返回 y.视图(输入形状[:-1])
```

请注意，当我们调用 Triton 内核时，通过传递元组 (cdiv(n_rows, ctx.<|endofcontentisolation>ROWS_TILE_SIZE),)。然后，我们可以通过 tl 访问线程块索引。程序ID(0)在我们的内核中。

反向传播由于我们正在定义自己的内核，因此还需要编写自己的反向函数。

在前向传播中，我们获得了层的输入，并需要计算其输出。而在反向传播中，回想一下，我们将收到目标函数关于输出的梯度，然后需计算该梯度相对于每个输入的导数。在本例中，我们的操作接收一个矩阵作为输入： x : Rn×h，以及一个权重向量： w : Rh。简而言之，我们把这一操作记为 f(x, w)，其输出范围是 Rn。接下来，假设我们已知 $\mathcal{V}$(x,w)L，即损失函数 L关于该层输出的梯度，那么我们可以应用多元链式法则，推导出关于 x 和 w的以下梯度表达式：

$$(\nabla_x \mathcal{L})_{ij} = \sum_{k=1}^{n} \frac{\partial f(x,w)_k}{\partial x_{ij}} (\nabla_{f(x,w)} \mathcal{L})_k = w_j \cdot (\nabla_{f(x,w)} \mathcal{L})_i \tag{2}$$

$$(\nabla_w \mathcal{L})_j = \sum_{i=1}^{n} \frac{\partial f(x,w)_i}{\partial w_j} (\nabla_{f(x,w)} \mathcal{L})_i = \sum_{i=1}^{n} x_{ij} \cdot (\nabla_{f(x,w)} \mathcal{L})_i \tag{3}$$

这提供了一个计算反向传播的简单公式。为了得到关于 x 的反向步骤，我们应用式（2），并取 w 与 $\mathcal{V}$(x,w) 的外积。而要计算关于 w 的反向步骤（即 ($\mathcal{V}$L)j），则需将输入梯度与对应的输出行相乘。

我们的反向传播内核将首先定义所有块指针，然后计算 $\mathcal{V}$L:

```
@triton.jit
def加权和反向传播(
        x_ptr，权重指针，  # 输入
        梯度输出指针，  # 梯度输入
        grad_x_ptr，partial_grad_weight_ptr，  # 梯度输出
        步长_xr，步长_xd，
        步幅_wd，
        步长_gr，
        stride_gxr，stride_gxd，
        stride_gwb, stride_gwd,
        行数，D，
        ROWS_TILE_SIZE：tl.常量，D_TILE_SIZE：tl.常量，
):
        行块索引 = tl.程序ID(0)
        n_行瓦片 = tl.程序数量(0)

        # 输入
        grad_output_block_ptr = tl.make_block_ptr(
            梯度输出指针，
            形状=(行数,), 步长=(步长格,),
            偏移量=(行块索引 * 行块大小,),
            块形状=(行块大小,),
            排序=(0,),
        )

        x_block_ptr = tl.make_block_ptr(
            x_ptr，
            形状=(行数, 维度,), 步幅=(x轴步幅, x维度步幅),
            偏移量=(行块索引 * 行块大小, 0),
            块形状=(行块大小, 列块大小),
            排序=(1, 0),
        )

        weight_block_ptr = tl.make_block_ptr(
            权重指针，
            形状=(D,), 步长=(stride_wd,),
            偏移量=(0,), 块形状=(D_TILE_SIZE,),
            排序=(0,),
        )

        grad_x_block_ptr = tl.make_block_ptr(
            梯度x指针，
            形状=(行数, 维度,), 步长=(跨行步长, 跨维度步长),
            偏移量=(行块索引 * 行块大小, 0),
            块形状=(行Tile尺寸, 列Tile尺寸),
            排序=(1, 0),
        )

        partial_grad_weight_block_ptr = tl.make_block_ptr(
            部分梯度权重指针，
            形状=(n_row_tiles, D,), 步幅=(stride_gwb, stride_gwd)
            偏移量=(行块索引, 0),
            块形状=(1, D_TILE_SIZE),
            排序=(1, 0),
        )

        for i in range(tl.cdiv(D, D_TILE_SIZE)):
            grad_output = tl.load(grad_output_block_ptr, boundary_che  k = (0,), padding_option="zero") # (ROWS_TILE_SIZE,)

            # grad_x 的外积
            权重 = tl.加载（权重块指针，边界检查=(0,)，p ）                dding_option="zero") # (D_TILE_SIZE,)
            grad_x_row = grad_output[:, None] * weight[None, :]
            tl.存储(grad_x_block_ptr, grad_x_row, 边界检查=(0,             1))

            # 尽可能减少 grad_weight 结果的行数                          t
```

```
            行 = tl.load(x_block_ptr, boundary_check=(0, 1), padding_option="zero") # (ROWS_TILE_SIZE, D_TILE_SIZE) grad_w
            eight_row = tl.sum(row * grad_output[:, None], axis=0, keep_dims=True)
            tl.store(偏导权重块指针, grad_weight_row, 边界检查=(1,)) # 始终不会超出第i²维的边界

            # 将指针沿D方向移动到下一个区块 x_block_ptr = x_block_ptr.advance((0, D_TILE_SIZE)) weight_bl
            ock_ptr = weight_block_ptr.advance((D_TILE_SIZE,)) partial_grad_weight_block_ptr = partial_gr
            ad_weight_block_ptr.advance((0, D_TILE_SIZE)) grad_x_block_ptr = grad_x_block_ptr.advance((
            0, D_TILE_SIZE))
```

计算梯度 V 非常简单，我们将结果写入输出张量的相应分块中。然而，计算 V 则稍显复杂。每个内核实例负责处理 x 的一行分块，但现在我们需要在 x 的 行上对 进行求和。为了简化反向传播过程中的这一求和操作，我们假设 partial_grad_weight_ptr 已包含一个 n_row_tiles × H 的矩阵，其中第一维仅在 x 的行分块内进行缩减。因此，我们先在当前行分块内完成缩减操作，再将结果写入该张量。而在内核外部，我们将使用 torch.sum 对 V 进行全局求和，以汇总各行分块的计算结果。最后， autograd.Function 的实现部分则相对简单：

```
类 加权求和函数(torch.autograd.函数）：
    @staticmethod
    def 前向传播(ctx, x, weight):
        # ...（先前已定义）

    @staticmethod
    def 反向传播(ctx, grad_out):
        x，权重 = 上下文.已保存的张量
        ROWS_TILE_SIZE, D_TILE_SIZE = 上下文.行块大小，上下文.D_TILE_SIZE # 这些不必相同 n_rows, D = x.形状


        # 我们的设计策略是，每个线程块首先将数据写入一个局部缓冲区，# 然后我们对这个缓冲区进行归约操作，以得到最
        终的梯度。
        partial_grad_weight = torch.empty((cdiv(n_rows, ROWS_TILE_SIZE), D), device=x.设备，数据类型=x.dtype) gr
        ad_x = torch.空置_like(x)

        加权求和反向传播[(cdiv(行数, 行块大小))](
            x，权重，
            梯度输出，
            梯度_x，权重的偏导数，
            x.步长(0)，x.步长(1)，
            权重.步长(0)，
            grad_out.stride(0)，
            grad_x.stride(0), grad_x.stride(1)，
            partial_grad_weight.stride(0), partial_grad_weight.stride(1), NUM_ROWS=n_rows, D=D,

        行块大小=行块大小，列块大小=列块大小，)

        grad_weight = 部分梯度权重.沿轴=0求和 返回 梯度x，梯度权重
```

终于，我们现在可以得到一个函数，其功能与在 torch.nn.functional 中实现的函数非常相似：

```
f_weightedsum = 加权求和函数.应用
```

现在，对两个 PyTorch 张量 x 和 w 调用 f_weightedsum，将得到如下所示的张量：

```
张量（[ 90.8563, -93.6815, -80.8884, ..., 103.4840, -21.4634, -24.0192], <
        设备='cuda:0'，梯度函数=<WeightedSumFuncBackward>)
```

请注意附加到张量上的grad_fn−−这表明当该张量出现在计算图中时，PyTorch 知道在反向传播阶段调用哪个函数。至此，我们已完成对加权求和操作的 Triton 实现。

1当然，我们也可以为此编写自己的内核。

1.3.2 FlashAttention-2 前向传播

您将用显著改进的Triton实现替换现有的PyTorch注意力机制，该实现遵循FlashAttention-2 [Dao, 2023]。FlashAttention-2采用了一些技巧，以分块方式计算前向传播，从而实现高效的内存访问模式，并避免了在全局内存中显式存储完整注意力矩阵的需求。

在进入本节之前，我们强烈建议您至少阅读一下原始的FlashAttention论文[Dao et al., 2022]，这将帮助您深入理解实现FlashAttention高效注意力机制的核心技术：即以分块（tile）为单位、在线计算Softmax函数（这一技术由[Milakov和Gimelshein, 2018]提出）。此外，我们也推荐您参阅He [2022]，以进一步了解GPU实际执行PyTorch代码时的工作原理。

理解普通注意力机制中的效率低下问题。 请记住，注意力机制的前向传播过程（暂不考虑掩码操作）可以表示为：

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top / \sqrt{d} \tag{4}$$

$$\mathbf{P}_{ij} = \mathrm{softmax}_j(\mathbf{S})_{ij} \tag{5}$$

$$\mathrm{O} = \mathrm{PV} \tag{6}$$

标准的反向传播是

$$\mathbf{dV} = \mathbf{P}^\top \mathbf{dO} \tag{7}$$

$$\mathbf{dP} = \mathbf{dO}\mathbf{V}^\top \tag{8}$$

$$\mathbf{dS}_i = \mathrm{dsoftmax}(\mathbf{dP}_i) = \left(\mathrm{diag}(\mathbf{P}_i) - \mathbf{P}_i\mathbf{P}_i^\top\right)\mathbf{dP}_i \tag{9}$$

$$\mathbf{dQ} = \mathbf{dS}\mathbf{K}/\sqrt{d} \tag{10}$$

$$\mathbf{dK} = \mathbf{dS}^\top\mathbf{Q}/\sqrt{d}, \tag{11}$$

正如我们所见，反向传播过程依赖于正向传播中生成的一些非常庞大的激活值。例如，在公式（7）中计算 dV 时，需要使用 P，即形状为 (batch_size, n_heads, seq_len, seq_len) 的注意力分数——这一激活矩阵的大小 随序列长度呈 二次方增长，这也正是我们在上述针对长序列长度的注意力机制基准测试中遇到内存问题的根本原因。在标准注意力机制的正向和反向传播过程中，为了在片上SRAM与GPU HBM之间传输 P及其他大型激活值，我们都付出了巨大的内存I/O开销。而在标准实现中，这样的数据传输操作通常会进行多次：比如，在标准的反向传播实现中，为了完成公式（7）和（9）中的计算，就需要从HBM中读取P。

FlashAttention 的主要目标是避免将注意力矩阵频繁地读写至 HBM，从而降低 I/O 开销和峰值内存成本。我们通过三种技术实现这一目标：分块处理、重新计算以及算子融合。

分块计算。 为避免将注意力矩阵频繁地读写到HBM中，我们采用一种无需访问完整输入即可完成softmax归一化的计算方法。具体而言，我们将注意力计算过程重新组织，将输入数据划分为多个小块，并对这些输入块进行多次遍历，从而逐步完成softmax归一化操作。

重新计算。 我们避免将形状为（batch_size, n_ 头, seq_len, seq_len）的大型中间注意力矩阵存储在HBM中。相反，我们会在HBM中保存一些“激活检查点”，然后在反向传播过程中重新计算前向传播的一部分，以获取计算梯度所需的其他激活值。此外，FlashAttention-2还存储了注意力分数的logsumexp，即 L，这将在在

用于简化反向传播的计算。该表达式为 L是:

$$L_i = \log\left(\sum_j \exp\left(\mathbf{S}_{ij}\right)\right) \tag{12}$$

在我们的最终内核中，我们将以在线方式计算这一过程，但最终结果应保持一致。通过结合分块处理与重新计算技术，我们的内存I/O和峰值使用量将不再依赖于 sequence_length2 ，因此我们可以采用更长的序列长度。

算子融合。 最后，我们通过在一个独立的内核中完成所有操作，避免了注意力矩阵及其他中间激活值的重复内存I/O——这被称为算子或内核融合。我们将为前向传播编写一个单独的Triton内核，该内核能在HBM与SRAM之间实现有限的数据传输，同时完成注意力机制涉及的所有计算操作。算子融合在一定程度上得益于重新计算技术，因为我们可以省去通常需要将每个中间激活值存储到HBM以供后续使用的额外内存I/O开销。

如需进一步了解这些技术的直观原理，请参阅FlashAttention相关论文[Dao et al., 2022, Dao, 2023]。

带重新计算的反向传播。 利用 L，我们可以高效地进行适当的重新计算并完成反向传播。在开始反向传播之前，我们预先将值 D = rowsum(O ◦dO)（其中 ◦表示逐元素乘法）存入全局内存，该值实际上等于 rowsum(P ◦dP)，因为 PdP = P(dOV) = (PV)dO = OdO （且对于任意矩阵 A 和 B，有 rowsum(A ◦B) = diag(AB)）。借助 L 和 D 向量，我们可以在不使用 softmax 的情况下完成反向传播。目前，反向传播的完整计算过程如下：

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top/\sqrt{d} \tag{13}$$
$$\mathbf{P}_{ij} = \exp\left(\mathbf{S}_{ij} - L_i\right) \tag{14}$$
$$\mathbf{dV} = \mathbf{P}^\top\mathbf{dO} \tag{15}$$
$$\mathbf{dP} = \mathbf{dO}\mathbf{V}^\top \tag{16}$$
$$\mathbf{dS}_{ij} = \mathbf{P}_{ij} \circ (\mathbf{dP}_{ij} - D_i) \tag{17}$$
$$\mathbf{dQ} = \mathbf{dS}\mathbf{K}/\sqrt{d} \tag{18}$$
$$\mathbf{dK} = \mathbf{dS}^\top\mathbf{Q}/\sqrt{d}, \tag{19}$$

我们可以看到，这一系列操作并不需要我们在前向传播过程中将注意力分数 P 存储在HBM中——我们只需根据(13)和(14)中的激活值 Q、K 以及 L重新计算它们。

FlashAttention前向传播的详细信息。现在，我们已经对FlashAttention-2中所用的技术有了大致了解，接下来将深入探讨您即将实现的FA2前向传播内核的具体细节。为了避免频繁地在HBM中读写注意力矩阵，我们希望采用分块策略，即独立计算输出矩阵的每个分块，互不干扰。为此，我们需要能够高效地计算出 P 的分块，理想情况下，这些分块应在查询和键两个维度上均采用分块形式。

然而，当我们对 S应用softmax时，需要先将 S的整行数据规约以计算softmax分母，这意味着我们无法直接以分块方式计算 P。FlashAttention-2通过使用在线softmax来解决这一问题。在接下来的文本中，我们将用下标索引 i表示当前的查询分块，用上标索引 (j)表示当前的键分块。沿查询维度的分块大小为 Bq，而沿键维度的分块大小则为 Bk。我们不会在隐藏维度 d上进行分块操作。

我们还保留了一些按行累积的值， m(j)i ∈ RBq 和 l(j)i ∈ RBq。其中，按行累积的 m(j)i 值是一个运行最大值，我们会持续跟踪它，以便以数值稳定的方式计算 Softmax（请回顾我们在作业 1 中实现 Softmax 时使用的这一技巧）。每当处理完 S 的新一行分块时（即当 j 增加时），我们将更新 m(j)i。通过使用运行最大值，我们可以计算出未归一化的 Softmax 值

(分子) 作为 ˜P(j)i = exp Sij − m(j)i 。 l(j)i 是 softmax 分母的实时代理，并将根据未归一化的 softmax 值进行更新，即 l(j)i = exp(m(j−1)i)。当我们最终输出结果时，还需通过使用 l(Tk)i 来完成归一化操作，其中 l(Tk)i 是处理完所有关键分块后 l(j)i 的最终值。算法 1 展示了应在 GPU 上实现的前向传播过程。

算法1 FlashAttention-2 前向传播 **要求：** Q ∈ RNq×d，K、V ∈ RNk×d，分块大小 Bq、Bk 将 Q分割为 Tq = NqBq — 瓷砖 Q1，...， QTq，尺寸为 Bq × d

拆分 K, V 为 Tk = NkBk 瓷砖 K(1)，...， K(Tk)，以及 V(1)，...， V(Tk)，其尺寸均为 Bk × d

for i = 1, ... , T**q do**加载 全局内存中的 Qi

初始化 O(0)i = 0 ∈ RBq×d,l(0)i = 0 ∈ RBq,m(0)i = −∞ ∈ R**Bq对于** j = 1, ... , T**k 执行**从全局内存加载 K(j), V(j)

计算预 softmax 注意力分数的分块 S(j)i = Qi(K(j)) / √d ∈ RBq×Bk

计算 m(j)i = 最大 (m(j−1)i , 行最大值 (S(j)i)) ∈ RBq

计算 ˜P(j)i = exp (S(j)i − m(j)i) ∈ RBq×Bk

计算 l(j)i = exp (m(j−1)i − m(j)i l(j−1)i + 行和 (˜P(j)i)) ∈ RBq

计算 O(j)i = 对角线 (exp(m(j−1)i − m(j)i) O(j−1)i + ˜P(j)i V(j)

结束 for

计算 Oi = 对角线(l(Tk)i)−1 O(Tk)i

计算 Li = m(Tk)i + log (l(Tk)i)

将 O 写入全局内存，作为 O 的第 i² 个分块。接着，将 Li 写入全局内存，作为 L **的第** i² 个分块。循环结束。

返回输出 O 和 logsumexp L.

在我们开始在Triton中实现前向传播之前，这里先总结一些编写Triton内核的通用技巧与窍门。

Triton 使用技巧与窍门

- 你可以在 Triton 中使用 print 语句，配合 tl。设备打印 用于调试： https://triton-lang.org/main/python-api/generated/triton.language.device_print.html。虽然我们发现该设置 TRITON_INTERPRET=1 可在 CPU 上运行 Triton 解释器，但目前仍存在一些问题。
- 在定义块指针时，务必确保它们具有正确的偏移量，并且块偏移量应乘以相应的瓦片尺寸。
- 线程块的启动网格已设置为

```
kernel_fn[(launch_grid_d1, launch_grid_d2, ...)](...arguments...)
```

在torch.autograd.Function子类的方法中，正如我们在加权求和示例中所看到的。

- 使用 tl.dot. 执行矩阵乘法。
- 要推进块指针，使用 *_block_ptr = *_block_ptr.advance(...)

问题 (快进)：15分

(a) 编写一个纯 PyTorch（不使用 Triton）的 autograd 函数，用于实现 FlashAttention-2 的前向传播。该函数将比常规的 PyTorch 实现慢得多，但有助于你调试 Triton 内核。

> 你的实现应接收输入 Q、K 和 V，以及一个标志位 is_causal，并生成输出 O 和 logsumexp 值 L。在此任务中，你可以忽略 is_causal 标志位。autograd.Function 的 forward 方法随后应保存 L、Q、K、V、O 以供反向传播使用，并返回 O。请记住，autograd.Function 类的 forward 方法实现始终将上下文作为其第一个参数。任何autograd.Function 类都需要
>
> 实现一个反向方法，但目前你可以先简单地抛出**NotImplementedError**。如果你需要一些对比基准，可以使用PyTorch实现式子4至6以及式子12，并将你的输出与之进行比较。
>
> 界面随后被定义为 **forward**(ctx, Q, K, V, is_causal=False)。请自行确定瓷砖大小，但务必确保其至少为 16 × 16。我们将始终使用恰好是2的幂且不低于16的尺寸来测试您的代码，因此您无需担心越界访问问题。
>
> **交付物**：一个Torch.Autograd.Function子类，用于在前向传播中实现FlashAttention-2。为测试您的代码，请实现 [adapters.get_flashattention_autograd_function_pytorch]. 然后，使用 uv run pytest -k test_flash_forward_pass_pytorch 运行测试，并确保你的实现能够通过该测试。

(b) 按照算法1，为FlashAttention-2的前向传播编写一个Triton内核。然后，编写另一个 torch.autograd.Function的子类，该子类在前向传播中调用此（融合）内核，而非直接在PyTorch中计算结果。以下是一些针对特定问题的提示：

- 为进行调试，我们建议将您执行的每个Triton操作的结果，与您在（a）部分编写的分块式PyTorch实现进行对比。
- 您的启动网格应设置为（Tq, batch_size），这意味着每个 Triton 程序实例将仅加载来自单个批次索引的元素，并且只读写 Q、O 和 L 的单个查询块。
- 内核应仅包含一个循环，该循环将遍历关键图块 $1 \le j \le Tk$。
- 在循环末尾向前推进块指针。
- 请使用以下函数声明（利用我们提供的块指针，您应能推断出其余指针的设置方式）：

```
@triton.jit
def flash_fwd_kernel(
    Q指针，K指针，V指针，
    O指针，L指针，
    stride_qb, stride_qq, stride_qd,
    步长_kb，步长_kk，步长_kd，
    stride_vb、stride_vk、stride_vd、
    stride_ob, stride_oq, stride_od,
    步长LB，步长LQ，
```

```
        查询数量，密钥数量，
        规模，
        D：tl.constexpr，
        Q_TILE_SIZE：tl.常量，
        K_TILE_SIZE：tl.常量，
):
        # 程序索引
        查询块索引 = tl.程序ID(0)
        批索引 = tl.程序ID(1)

        # 将每个指针与对应的批次索引偏移
        # 与每个张量的批步长相乘
        Q_block_ptr = tl.make_block_ptr(
            Q_ptr + 批次索引 * 增量_qb，
            形状=(N_QUERIES, D)，
            步长=(stride_qq, stride_qd)，
            偏移量=(查询瓦片索引 * Q瓦片尺寸, 0)，
            块形状=(Q_TILE_SIZE, D)，
            排序=(1, 0)，
        )

        ...
```

哪里尺度是 $1/\sqrt{d}$，而 Q_TILE_SIZE 和 K_TILE_SIZE 分别是 Bq 和 Bk。你可以稍后对这些进行调优。

这些附加指南可能有助于您避免精度问题：

- 片上缓冲区（Oi,l, m）应具有数据类型 tl。如果需要累积到输出缓冲区，请使用累加参数 acc（acc = tl.点积(..., acc=acc)）。
- 在相乘之前，将 ˜P(j)i 转换为 V(j) 的 dtype；同时，在将其写入全局内存前，将 Oi 转换为适当的 dtype。转换操作通过 tensor.to 完成。你可以使用 tensor.dtype 获取张量的 dtype，而通过 *_block_ptr.type.element_ty 则可获取块指针或普通指针的元素类型。

交付物：一个继承自torch.autograd.Function的子类，用于在前向传播中通过您的Triton内核实现FlashAttention-2。请实现
[adapters.get_flash_autograd_function_triton]. 然后，使用 uv run pytest -k test_flash_forward_pass_triton 运行测试，并确保你的实现能够顺利通过。

(c) 在你的因果关系 自动微分函数实现中，将一个标志作为最后一个参数添加到 的 autograd.Function 实现中。
掩码。这应是一个布尔标志，当设置为 True时，启用因果掩码的索引比较；反之，则禁用。您的Triton内核应相应地增加一个额外的参数
is_causal: tl.constexpr（这是必需的类型注解）。在 Triton 中，为查询和键构建适当的索引向量，并将它们进行比较，以形成一个大小为Bq × Bk 的方形掩码。对于被屏蔽的元素，将其值设置为常量^-1e6。
注意力分数矩阵的对应元素 S
向后使用 ctx.is_causal = is_causal.

(j)
i . Make sure to save the mask flag for

Deliverable：为您的 `torch.autograd.Function` 子类添加一个额外的标志，用于实现带有因果掩码的 FlashAttention-2 前向传播，并采用您的 Triton 内核。请确保该标志为可选，默认值设为 `False`，以保证现有测试仍能通过。

通过重新计算实现反向传播 注意，与公式7至11中的标准反向传播不同，我们可以通过重新计算来避免公式13至19所示反向传播中的Softmax操作。这意味着我们可以使用一个简单的内核来完成反向传播的计算，而无需任何在线技巧。因此，在这一部分，你可以通过调用 torch 来实现反向传播。

PyTorch 函数（非 Triton）。

问题（闪回后移）：5分

使用PyTorch（而非Triton）和 torch.，实现你的FlashAttention-2自动微分的反向传播过程 autograd。你的实现应接收 Q、K、V、O、dO以及 L张量作为输入，并返回 dQ、dK和 dV。请务必计算并利用 D向量，计算过程可参考式子13至19。

可交付成果：要测试您的实现，请运行 uv run pytest -k test_flash_backward.

现在，让我们比较一下你（部分地）使用Triton实现的FlashAttention-2与你用PyTorch实现的常规Attention的性能。

问题（闪存基准测试）：5分

(a) 使用 Triton 编写一个基准测试脚本，并运行do_bench 测试，以对比您实现的（部分）Triton 版本的 FlashAttention-2 前向和反向传递操作与标准 PyTorch 实现（即未使用 FlashAttention 的版本）之间的性能差异。

具体而言，您将提交一张表格，其中包含您的Triton和PyTorch实现方案在前向、反向以及端到端前向-反向传递过程中的延迟数据。在开始基准测试之前，随机生成所需输入，并仅使用单个H100 GPU进行测试。始终采用批大小为1，并启用因果掩码。测试范围涵盖从128到65536的各次幂2的序列长度的笛卡尔积，嵌入维度大小则覆盖从16到128的各次幂2的数值，同时测试精度分别为 torch. bfloat16 和 torch. float32。根据输入规模的不同，您可能需要调整分块大小。

交付物：一份结果表格，用于比较您实现的FlashAttention-2与PyTorch实现，在上述设置下分别报告前向、反向及端到端延迟。

1.3.3 FlashAttention-2 排行榜

第2次作业的排行榜将测试你实现FlashAttention-2的速度（包括前向和反向传播两部分）。我们鼓励你进一步优化自己的实现，尽情发挥创意，运用一切可行的技巧来提升性能。但请注意，你不得更改函数的输入输出接口，并且必须使用Triton（很遗憾，不能使用CUDA）。测试时，输入数据将以BF16格式提供，并采用因果掩码；你的实现还需通过与常规实现完全一致的测试。此外，代码必须是你自己独立完成的，严禁直接引用或使用已有的现成实现。计时将基于H100 GPU，在批次大小为1、查询、键和值的序列长度均为16,384，以及模型维度d=1024、包含16个注意力头的样本上进行。最终，我们将对排名前5到10位的提交作品进行正确性和性能的双重验证。用于计时测试的具体实现代码如下：

```
def test_timing_flash_forward_backward():
    n_heads = 16
    d_head = 64
    序列长度 = 16384
    q, k, v = torch.randn(
    3, n_heads, sequence_length, d_head, device='cuda', dtype=torch.bfloat16, requires_grad=Tr
    ue )

    flash =  torch.compile(FlashAttention2.apply)
```

```
    def 快速前后向传播():
        o = 闪存(q, k, v, True)
        损失 = o.求和()
        损失.反向传播()

    结果 = 特里同.测试.do_bench(前向后向传播, 重复次数=10000, 预热次数=1000) 打印结
    果(results)
```

出于测试目的，您可以将重复次数和预热时间（以毫秒为单位）缩短至更短的值。以下是一些改进建议：

- 为你的内核调整线程块大小（可使用 Triton 自动调优功能！）
- 调整额外的 Triton 配置参数
- 在Triton中实现反向传播，而不仅仅是 torch.compile（参见下文1.3.4节）
- 在反向传播过程中，对输入数据进行两次遍历：一次用于计算 dQ，另一次用于计算 dK 和 dV，以避免块之间的原子操作或同步问题。
- 在执行因果掩码时，提前停止程序实例，跳过所有始终全为零的图块。
- 将未屏蔽的图块与图块对角线分离，前者无需比较索引即可完成计算，而后者只需一次比较。
- 在H100上使用TMA（张量内存加速器）功能，操作步骤可参考本教程中的类似模式。

将您的最佳成绩提交至排行榜：

github.com/stanford-cs336/assignment2-systems-leaderboard

1.3.4 可选：Triton反向传播

如果你希望进一步练习Triton，或想快速提交排行榜结果，我们提供了下面的分块FlashAttention-2反向传播实现，你可以直接在Triton中加以实现。算法2展示了应在Triton中实现的FlashAttention-2反向传播流程。这里的一个关键技巧是：将P计算两次——一次用于 dQ的反向传播，另一次则用于 dK和 dV的反向传播。这样，我们便能避免线程块间的同步操作。

算法2 分块FlashAttention-2反向传播步骤 要求：

Q, O, dO ∈ R × , K, V ∈ R × , L ∈ R , 分块大小 Bq, Bk

计算 D = 行和(dO ○O) ∈ R

将Q、O、dO分割为Tq = $\frac{Nq}{Bq}$ 瓷砖 Q1、…、QTq，O1、…、OTq，dO1、…、dOTq，每块尺寸均为 Bq × d

将 K、V 分割为 Tk = $\frac{恩克}{Bk}$ 瓷砖 K(1),⋯, K(Tk) 和 V(1),⋯, V(Tk)，每块尺寸为 Bk × d

将L、D分割为Tq个大小均为Bq的子块L1, …, LTq 和 D1, …, DTq，其中j = 1, …, Tk时

从全局内存中加载 K(j) 和 V(j)

初始化 dK(j) = dV(j) = 0 ∈ RBk×d

对于 i = 1, …, Tq 做

从全局内存中加载Qi、Oi、dOi和dQi

计算注意力分数矩阵 S $^{(j)}_{i}$ = $\frac{Qi(K(j))\sqrt{}}{\overline{d}}$ ∈ RBq×Bk

计算注意力概率 P $^{(j)}_{i}$ = exp($S^{(j)}_{i}$ − 李） ∈ RBq×Bk

计算 dV(j) += (P $^{(j)}_{i}$) dOi ∈ RBk×d

计算 dP $^{(j)}_{i}$ = dOiV $_{j ∈ RBq×Bk}$

计算 dS $^{(j)}_{i}$ = P $^{(j)}_{i}$ ○($dP^{(j)}_{i}$ − Di) /√ $\overline{d}$ ∈ RBq×Bk

从全局内存中加载 dQi，然后更新 dQi += dS $^{(j)}_{i}$ K(j) ∈ RBq×d，并将其写回全局内存。为确保正确性，必须是原子操作！

计算 dK(j) += (dS $^{(j)}_{i}$) Qi ∈ R(Bk×d)。

结束 for

将dK(j)和dV(j）写入全局内存，作为dK和dV的第j个分块。结束循环

返回 dQ、dK、dV。

2 分布式数据并行训练

在本次作业的下一环节中，我们将探讨如何利用多块GPU来训练语言模型，重点聚焦于数据并行方法。首先，我们会简要介绍PyTorch中的分布式通信机制；随后，我们将研究一种简单的分布式数据并行训练实现方案，并在此基础上逐步实施和评估多种优化措施，以提升通信效率。

2.1 PyTorch 中的单节点分布式通信

让我们从一个简单的 PyTorch 分布式应用开始，其目标是生成四个随机整数张量，并计算它们的和。
在下面的分布式场景中，我们将启动四个工作进程，每个进程生成一个随机整数张量。为了在各工作进程间对这些张量进行求和，我们将调用**全归约**集体通信操作，该操作会将每个进程中原有的数据张量替换为经过全归约处理后的结果（即求和结果）。

现在让我们来看一些代码。

```
import os
我是朋友特别喜欢编程语和数据
库，对系统架构也十分感兴趣。
我是程序员。通过设计和实现一系列技术解决方案，<<
我是一个有趣的人。目前正在学习计算机科技，特别感兴趣于软件工程和系统架构。

def setup(rank, world_size):
        os.environ["MASTER_ADDR"] = "localhost"
        os.environ["MASTER_PORT"] = "29500"
        dist.init_process_group("gloo", rank=rank, world_size=world_size)

def distributed_demo(rank, world_size):
        设置（rank, world_size）
        ```pythondata = torch.randint(0, 10, (3,))```
        print(f"rank {rank} 数据（在执行all-reduce之前）： {data}")
        dist.all_reduce(data, async_op=False)
        print(f"rank {rank} 数据（经全归约后）： {data}")

if __name__ == "__main__":
        世界规模 = 4
        mp.spawn(fn=distributed_demo, args=(world_size,), nprocs=world_size, join=True)
```

运行上述脚本后，我们得到以下输出。正如预期的那样，每个工作进程最初都持有不同的数据张量。在执行了全局归约操作——即在所有工作进程中对张量进行求和之后，数据在各个工作进程中的原地被修改，以存储经过全局归约处理后的结果。2

```
$ uv run python distributed_hello_world.py
rank 3 data tensor([3, 7, 8])
0号进程的数据（在all-reduce之前）： tensor([4, 4, 7])
2号排名数据（在all-reduce之前）： tensor([6, 0, 7])
排名1的数据（在all-reduce之前）： tensor([9, 5, 3])
rank 1 data (after all-reduce):1^ tensor([22, 16, 25])
进程 0 的数据（All-reduce 后）： tensor([22, 16, 25])
```

2如果你多次运行此脚本，会发现打印输出的顺序并非确定性。由于该应用程序是在分布式环境中运行的，我们无法控制命令执行的确切顺序——我们唯一能保证的是：在 all-reduce 操作完成后，各个进程将持有位完全相同的结果张量。
```

8 全归约后第3位数据：tensor([22, 16, 25])
9 汇总后排名2的数据（经all-reduce操作）：tensor([22, 16, 25])

现在，让我们更仔细地回顾一下上面的脚本。命令 mp.spawn 会启动 nprocs 个进程，每个进程都会运行 fn，并传入提供的 args。此外，函数 fn 的调用方式为 fn(rank, *args)，其中 rank 是工作进程的索引（一个介于 0 和 nprocs-1 之间的值）。因此，我们的 distributed_demo 函数必须将这个整数 rank 作为其第一个位置参数来接受。同时，我们还传递了 world_size，它表示工作进程的总数。

这些工作进程各自属于一个 进程组，该进程组通过 dist.init_process_group 初始化。进程组代表多个将通过共享主节点进行协调与通信的工作进程。主节点由其 IP 地址和端口定义，且主节点运行着 rank 为 0 的进程。诸如 all-reduce 等集体通信操作会作用于进程组中的每个进程。

在本例中，我们使用了 “gloo” 后端初始化了进程组，但也有其他后端可供选择。特别是，“nccl” 后端将调用NVIDIA NCCL集体通信库，通常情况下，这对于CUDA张量的性能表现更为出色。不过，NCCL仅能在配备GPU的机器上使用，而Gloo则可在纯CPU机器上运行。一个实用的经验法则是在分布式GPU训练中使用NCCL，而在分布式CPU训练和/或本地开发时则选用Gloo。在本示例中，我们采用了Gloo，因为它支持在纯CPU机器上进行本地执行和开发。

在运行多GPU任务时，请确保不同进程使用不同的GPU。一种实现方法是调用 torch.cuda.在 setup 函数中，使用 set_device(rank))，这样 张量 .to("cuda") 就会自动将其移动到指定的设备上。或者，你也可以显式地为每个 rank 创建一个设备字符串（例如， device = f"cuda:{rank}"），然后将此设备字符串用作任何数据移动的目标设备（例如， 张量 .to(f"cuda:{rank}")）。

术语。 在本次作业的其余部分（以及你可能在网上看到的各种其他资源）中，你可能会遇到以下与PyTorch分布式通信相关的术语。尽管本作业将重点介绍单节点多进程的分布式训练，但这些术语对于理解分布式训练的整体概念同样具有帮助。请参阅图2以获取直观展示。

节点： 网络中的一个设备。

工作节点： 参与分布式训练的程序实例。在本次作业中，每个 worker 将拥有单个进程，因此我们将 worker、process 和 worker process 这些术语交替使用。然而，一个 worker 可能会使用多个进程（例如，用于加载训练数据），因此在实际应用中，这些术语并不总是等价的。

世界大小： 进程组中总工作线程的数量。

全局排名： 一个整数ID（介于0和world_size-1之间），用于唯一标识进程中的工作节点组。例如，对于大小为2的世界，一个进程的全局秩为0（主进程），另一个进程的秩为1。

本地世界大小： 在跨不同节点运行应用程序时，本地世界大小是指**的个数**在给定节点上本地运行的工作者。例如，如果我们有一个应用程序，在两个节点上各启动4个工作者，则全局工作器数量为8，而本地工作器数量则为4。需要注意的是，当应用仅运行于单个节点时，某个工作者的本地工作器数量等同于（全局）工作器总数。

本地排名： 一个整数ID（介于0和本地总进程数-1之间），用于唯一标识某个的索引；机器上的本地工作进程。例如，如果我们有一个应用程序，在两个节点上各启动了4个进程，那么每个节点将拥有本地排名分别为0、1、2和3的工作进程。需要注意的是，当运行单节点的多进程分布式应用时，进程的本地排名与其全局排名是等价的。

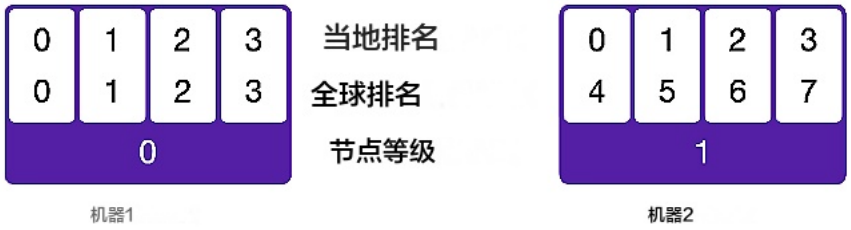

图2：一个分布式应用的示意图，该应用运行在2个节点上，世界规模为8。每个工作进程通过全局排名（从0到7）和本地排名（从0到3）进行标识。图片来源：lightning.ai/docs/fabric/stable/advanced/distributed_communication.html

2.1.1 分布式应用基准测试的最佳实践　　条款

在本次作业的这一部分，你将对分布式应用进行基准测试，以更深入地理解通信带来的开销。以下是一些最佳实践：

- 尽可能在同一台机器上运行基准测试，以方便进行受控比较。
- 在计时目标操作之前，需执行若干次预热步骤。这一点对于NCCL通信调用尤为重要。通常情况下，进行5轮预热迭代已足够。
- 调用 torch.cuda.synchronize() 用于在 GPU 上进行基准测试时，等待 CUDA 操作完成。请注意，即使调用通信操作且将 async_op 设置为**False**（此时操作仅在 GPU 上被排队，并不会等待通信真正完成），也仍需调用 synchronize()，因为该方法会确保所有已提交的 CUDA 任务均已完成。
- 不同Rank之间的计时可能会略有差异，因此通常会跨Rank汇总测量结果，以提高估算的准确性。您可能会发现，使用all-gather集合操作（特别是 dist.all_gather_ object函数）有助于从所有Rank收集结果。
- 通常情况下，先在 CPU 上使用 Gloo 进行本地调试，然后根据具体问题的需求，再切换到 GPU 上用 NCCL 进行性能基准测试。在不同后端之间切换时，只需修改 init_process_group 调用，并调整张量的设备类型转换即可。

问题（分布式通信单节点）：5分

编写一个脚本，用于在单节点多进程设置中对 all-reduce 操作的运行时进行基准测试。上述示例代码可作为合理的起点。请尝试调整以下设置：

后端 + 设备类型： Gloo + CPU，NCCL + GPU。

All-reduce 数据大小： 浮点数32位数据张量，范围涵盖1MB、10MB、100MB 和 1GB。

进程数量： 2、4 或 6 个进程。

资源需求：最多使用6个GPU。每次基准测试运行应少于5分钟。

3更多详情请参见 github.com/pytorch/pytorch/issues/68112#issuecomment-965932386。

交付物：绘制对比不同设置的图表和/或表格，并附上2-3句评论，阐述你的结果以及对各因素之间相互作用的思考。

2.2 A 纳　分布式数据并行T的直观实现　　训练

现在我们已经了解了在PyTorch中编写分布式应用的基本知识，接下来让我们构建一个分布式数据并行（DDP）训练的最小化实现。

数据并行将批次分割到多个设备（例如GPU）上，从而支持在超出单个设备容量的大批量尺寸下进行训练。例如，假设有四台设备，每台设备最多可处理32的批次尺寸，那么采用数据并行训练后，实际可用的批次尺寸将达到128。

以下是进行分布式数据并行训练的简单步骤：首先，每台设备会构建一个（随机初始化的）模型。接着，我们使用广播集体通信操作，将模型参数从Rank 0节点发送至所有其他节点。在训练开始时，每台设备都持有一份完全相同的模型参数和优化器状态副本（例如，Adam优化器中累积的梯度统计信息）。

1. 给定一个包含 n个样本的批次，该批次会被分片，每个设备将接收到 n/d 个互不重叠的样本（其中 d 是用于数据并行训练的设备数量）。 n 必须能被 d 整除，因为训练时间受限于最慢的进程。

2. 每个设备使用其本地存储的模型参数，在自身接收到的 n/d 个样本上执行前向传播，并通过反向传播计算梯度。需要注意的是，此时每个设备持有的正是基于其所接收的 n/d 个样本计算出的梯度。

3. 然后，我们使用全归约集体通信操作，在不同设备间对梯度进行平均，使得每个设备都持有基于所有 n 个样本计算得到的平均梯度。

4. 接下来，每台设备都会运行一个优化器步骤，以更新其自身的参数副本——从优化器的角度来看，这实际上只是在优化本地模型。由于所有设备均基于相同的初始模型和优化器状态启动，并且在每次迭代中都采用相同的平均梯度，因此各设备上的参数和优化器状态将始终保持同步。至此，我们已完成一次完整的训练迭代，可以重复这一过程。

问题（朴素DDP）：5分

交付物：编写一个脚本，通过在反向传播后对各个参数梯度进行全归约操作，以朴素方式实现分布式数据并行训练。为验证您的DDP实现是否正确，请使用该脚本在随机生成的数据上训练一个小型玩具模型，并确保其权重与单进程训练的结果完全一致。a

a如果你在编写此测试时遇到困难，不妨查看tests/test_ddp_individual_parameters.py

问题（naive_ddp_benchmarking）：3分

在这个朴素的DDP实现中，参数在每次反向传播后都会在各rank之间逐一进行全归约操作。为了更深入地理解数据并行训练的开销，可编写一个脚本来对先前实现的语言模型进行基准测试，该模型采用这种朴素的DDP实现进行训练。测量每步训练的总耗时，以及其中用于梯度通信的时间占比。实验将在单节点设置下（1个节点 × 2块GPU）针对§1.1.2中描述的XL模型尺寸进行数据收集。

可交付成果：一个代 您的基准测试设置说明，以及所测得的时间　　模型训练

迭代次数以及针对每种设置通信梯度所花费的时间。

2.3 改进最小DDP实现 注释

我们在§2.2中看到的最小DDP实现存在几个关键局限性：

1. 它对每个参数张量单独执行一次全归约操作。每次通信调用都会带来开销，因此，将多个通信调用批量处理以尽量减少这种开销可能是有利的。

2. 它会等待反向传播完成后再进行梯度通信。然而，反向传播是逐步计算的。因此，当某个参数的梯度准备就绪时，即可立即开始通信，而无需等待其他参数的梯度。这种机制使我们能够将梯度通信与反向传播的计算过程重叠起来，从而有效降低分布式数据并行训练的开销。

在本次作业的这一部分，我们将逐一解决这些限制，并衡量它们对训练速度的影响。

2.3.1 减少通信调用次数

与其为每个参数张量单独发起通信调用，不如尝试通过批量执行全归约操作来提升性能。具体来说，我们将要进行全归约的梯度拼接成一个单一的张量，然后在所有进程间对这个合并后的梯度执行全归约操作。在此过程中，使用torch._utils._flatten_dense_tensors 和 torch._utils._unflatten_dense_tensors 可能会有所帮助。

问题（最小化DDP平面基准测试）：2分

请修改你的最小化DDP实现，使其能够以展平梯度的形式从所有参数中通信张量。然后，在先前使用的条件下（1个节点 × 2块GPU，如§1.1.2所述的XL模型尺寸），将该实现的性能与原先为每个参数张量执行全归约操作的最小化DDP实现进行对比。

可交付成果：在采用单次批量All-Reduce调用进行分布式数据并行训练时，测量的每轮训练耗时以及梯度通信所花费的时间。用1-2句话比较批量处理与逐个梯度通信时的结果。

2.3.2 基于单个参数梯度的计算与通信重叠

尽管批量处理通信调用有助于降低发起大量小型全归约操作所带来的开销，但整个通信时间仍会直接增加系统负担。为解决这一问题，我们可以利用这样一个观察：在反向传播过程中，模型各层的梯度是逐步计算的（从损失函数开始，逐层向输入端推进）——因此，一旦某一层的参数梯度准备就绪，即可立即执行全归约操作，从而通过将反向传播的计算与梯度通信重叠，有效减少数据并行训练的开销。

我们将首先实现并基准测试一种分布式数据并行封装器，该封装器会在反向传播过程中，当各个参数张量准备就绪时，异步地执行全部归约操作。以下指针可能对您有所帮助：

反向钩子 若要在反向传播过程中，于参数的梯度累积完成后自动调用某个函数，可以使用 register_post_accumulate_grad_hook 函数。4

4更多信息及使用示例，请参阅 pytorch.org/docs/stable/generated/torch.Tensor.register_post_accumulate_grad_hook.html。

异步通信 所有 PyTorch 集体通信操作均支持同步（async_op=False）和异步执行（async_op=True）。同步调用会阻塞，直到集体操作已成功排队到 GPU 上。但这并不意味着 CUDA 操作已完成，因为 CUDA 操作本身是异步的。不过，后续使用该输出的函数调用将按预期行为执行。5 相比之下，异步调用会返回一个分布式请求句柄——因此，当函数返回时，并不能保证集体通信操作已成功排队到 GPU，更不用说已完成。若要等待操作成功排队到 GPU（从而确保输出可在后续操作中使用），可对返回的通信句柄调用 handle.wait()。

例如，以下两个示例分别通过同步或异步调用对张量列表中的每个张量执行 all-reduce 操作：

```
张量 = [torch.rand(5) for _ in range(10)]

# 同步，阻塞直至操作被排队到 GPU 上。
对 张量 中的 张量：
    dist.all_reduce(张量, 异步操作=False)

# 异步，每次调用后立即返回，并
# 最后等待结果。
处理器 = []
对于 张量 in 张量：
    句柄 = 距离.all_reduce(张量, 异步操作=True)
    句柄.追加（句柄）

# ...
# 可能会执行其他不依赖于all_reduce结果的命令
# ...

# 确保 all-reduce 调用已排队
# 因此，其他操作取决于
# 全归约输出可以被排队。
用于 处理 中的 处理程序：
    处理.等待()
处理. 清除()
```

问题（ddp_overlap_individual_parameters）：5分

实现一个Python类，用于处理分布式数据并行训练。该类应封装任意PyTorch的 nn.Module，并在训练前负责广播权重（确保所有进程具有相同的初始参数），同时发出通信调用来完成梯度的平均化操作。我们建议采用以下公共接口：

def __init__(self, module: torch.nn.Module): 给定一个已实例化的 PyTorch nn.Module，其将成为并行化，构建一个DDP容器，以处理跨进程的梯度同步。

def forward(self, *inputs, **kwargs): 调用包装模块的 forward() 方法，并传入提供了位置参数和关键字参数。

def finish_gradient_synchronization(self): 调用时，等待异步通信完成

5在高级场景中，如果你正在使用多个CUDA流，可能需要跨流进行显式同步，以确保输出已准备好供后续操作使用。详情请参阅 pytorch.org/docs/stable/notes/cuda.html#cuda-streams。

调用将被排队等待GPU处理。

要使用该类进行分布式训练，我们将为其传递一个需要包装的模块，然后在调用 finish_gradient_synchronization() 之前，再执行 optimizer.step()，以确保依赖于梯度的优化器步骤能够被正确排队：

```
模型 =  ToyModel()..to(device)
ddp_model = DDP(模型)

for _ 在 范围(训练步数):
    x, y = get_batch()
    logits = ddp_model(x)
    损失 = 损失函数(logits, y)
    损失.反向传播()
    ddp_model.完成梯度同步() 优化器.执行步骤()
```

可交付成果：实现一个容器类，以支持分布式数据并行训练。该类应能同时重叠梯度通信与反向传播的计算过程。为测试您的DDP类，首先实现适配器 [adapters.get_ddp_individual_parameters]和[adapters.ddp_individual_parameters_on_after_backward]（后者为可选步骤，具体是否需要取决于您的实现方案）。

然后，要执行测试，请运行 uv run pytest tests/test_ddp_individual_parameters.py。我们建议多次运行测试（例如5次），以确保测试能够稳定通过。

问题（ddp_overlap_individual_parameters_benchmarking）：1分

(a) 对您的DDP实现进行性能基准测试，重点考察在反向传播计算与各参数梯度通信重叠时的表现。将其性能与我们先前研究的几种设置（即针对每个参数张量执行全归约操作，或仅对所有参数张量的拼接结果进行一次全归约）进行对比，且采用相同的配置：1个节点、2块GPU，以及§1.1.2中描述的XL模型规模。

可交付成果：　当重叠反向传播时，每次训练迭代的测量时间
通过传递各个参数的梯度，并用1到2句话比较结果。

(b) 使用1个节点、2个GPU以及XL模型规模的配置，对您的基准测试代码进行插桩（instrumentation）Nsight 分析器，对比了最初的 DDP 实现与这种同时重叠反向计算和通信的 DDP 实现。通过可视化方式比较两条轨迹，并提供一张分析器截图，直观展示一种实现将计算与通信重叠，而另一种则未实现这一优化。

可交付成果：2张截图（一张来自最初的DDP实现，另一张来自本次DDP实现，该实现将计算与通信重叠），直观展示通信是否与反向传播过程重叠。

2.3.3 过载　　基于分桶并行的通信计算　　偏导数

在前一节（§2.3.2）中，我们已将反向传播的计算与各参数梯度的通信过程进行了重叠。然而，我们之前观察到，通过批量发送通信调用可以显著提升性能，尤其是在存在大量参数张量的情况下（这在深度Transformer模型中尤为常见）。我们先前尝试的批量发送方式是将所有梯度一次性发出，但这需要等待反向传播完成

传递以完成。在本节中，我们将尝试兼顾两全其美：首先将参数分组到多个桶中（从而减少总的通信次数），然后在每个桶中的所有组成张量准备就绪时执行全局规约操作（使我们能够实现通信与计算的重叠）。

问题（ddp_overlap_bucketed）：8分

实现一个Python类，用于处理分布式数据并行训练，通过梯度分桶技术提升通信效率。该类应封装任意输入的PyTorch nn.Module，并在训练前负责广播权重（确保所有进程拥有相同的初始参数），同时发出分桶化的通信调用以完成梯度平均。我们推荐如下接口：

def __init__(self, module: torch.nn.模块，桶大小（MB）：浮点数）： 给定一个已实例化的PyTorch nn.Module 要实现并行化，需构建一个 DDP 容器，该容器将负责跨进程的梯度同步。梯度同步应采用分桶机制，每个桶最多可容纳 bucket_size_mb 的参数量。

def forward(self, *inputs, **kwargs): 调用包装模块的 forward() 方法，并传入提供了位置参数和关键字参数。

def finish_gradient_synchronization(self): 调用时，等待异步通信完成调用将被排队等待GPU处理。

除了新增一个bucket_size_mb初始化参数外，此公共接口与我们之前采用逐个通信各参数的DDP实现完全一致。我们建议按照model.parameters()的逆序为参数分配到各个桶中，因为反向传播过程中，梯度大致会以该顺序就绪。

交付物：实现一个容器类，用于处理分布式数据并行训练。该类应支持梯度通信与反向传播计算的重叠执行。此外，梯度通信需采用分桶机制，以减少总的通信调用次数。为验证您的实现，请完成 [adapters.get_ddp_bucketed]、[adapters.ddp_bucketed_on_after_backward]，以及 [adapters.ddp_bucketed_on_train_batch_start]（后两者为可选步骤，具体是否需要取决于您的实现方案）。

然后，要执行测试，请运行 pytest tests/test_ddp.py。我们建议多次运行测试（例如5次），以确保测试能够稳定通过。

问题（ddp_分桶基准测试）：3分

(a) 使用与先前实验相同的配置（1个节点，2块GPU，XL模型尺寸），测试您分桶实现的基准性能，并逐步调整最大分桶大小（1 MB、10 MB、100 MB、1000 MB）。将这些结果与未采用分桶技术的先前实验进行对比——结果是否符合您的预期？如果存在偏差，原因何在？必要时，可借助PyTorch性能分析器，深入理解通信调用的执行顺序和具体实现方式。此外，您认为应如何调整实验设置，才能使最终结果与预期一致？

可交付成果：针对不同桶大小，测量的每次训练迭代所需时间。请用3到4句话对结果、你的预期，以及任何不匹配现象的潜在原因进行简要评述。

(b) 假设为一个桶计算梯度所需的时间与它所需的时间完全相同。用于传递梯度分桶。请写出一个方程，以函数形式建模DDP的通信开销（即反向传播完成后额外花费的时间）。

模型参数总大小（字节）（s），All-Reduce算法带宽（w，计算方法为每个Rank的数据量除以完成All-Reduce操作所需的时间），每次通信调用的开销（秒）（o），以及分桶数量（nb）。基于此公式，推导出一个能最小化DDP开销的最优分桶大小公式。

可交付成果：用于建模DDP开销的方程，以及最优桶大小的计算公式。

2.4 4D 并行性

尽管实现起来复杂得多，但我们仍可在更多维度上并行化训练过程。最常见的是讨论五种并行化方法：

- **数据并行（DP）** – 将数据批次分割到多个设备上，每个设备为其所属的批次计算梯度。随后，这些梯度需在各设备间进行某种方式的平均。
- **全局分片数据并行（FSDP）** – 优化器状态、梯度和权重在各个设备间进行拆分。如果仅使用数据并行（DP）和全局分片数据并行，每个设备都需要先从其他所有设备收集到权重分片，才能执行前向或反向传播操作。
- **张量并行（TP）** – 激活值按新维度进行分片，每个设备仅计算其对应分片的输出结果。在张量并行中，我们可选择沿操作的输入或输出进行分片。若将权重与激活值分别按相应维度分片，则张量并行可与FSDP高效协同使用。
- **管道并行（PP）** – 模型按层被分割成多个阶段，每个阶段运行在不同的设备上。
- **专家并行（EP）** – 我们将专家模型中的各个专家分配到不同的设备上，每个设备独立计算其对应专家的输出结果。

通常，我们总是将FSDP与TP结合使用，因此可以将其视为单一的并行化维度。这样一来，我们就有了4个并行化维度：DP、FSDP/TP、PP和EP。此外，我们还将专注于稠密模型（而非MoE），因此不再深入讨论EP。

在讨论分布式训练时，我们通常会将集群描述为一个由设备构成的网格，其中网格的各个轴正是我们定义并行化方式所依据的维度。例如，如果我们拥有16块GPU，而模型规模又远超单个设备的承载能力，那么我们可能会倾向于将网格组织成一个 4×4 的GPU阵列，其中第一维代表数据并行（DP），第二维则同时结合了全参数并行（FSDP）与张量并行（TP）。

有关这些方法的工作原理及其通信和内存开销的推导，更多细节请参阅《TPU扩展手册》（Austin 等，2025 年）第 5 部分的概述——这将特别有助于解决下述问题。如需更深入的流水线并行讨论，请参考《超大规模实践手册》附录（Nouamane Tazi，2025 年）。此外，本书的其余部分还包含大量其他可能对您有所帮助的信息。

问题（通信计费）：10分

考虑一种新的模型配置XXL，其参数设置如下： d_model=16384，d_ff=53248，以及 num_blocks=126。由于对于超大规模模型而言，绝大部分浮点运算操作都集中在前馈网络中，我们做出了一些简化假设。首先，我们省略了注意力机制、输入嵌入层以及输出线性层。接着，我们假定每个前馈网络仅由两个线性层构成（忽略激活函数），其中第一个线性层的输入维度为 d_model，输出维度为 d_ff；第二个线性层的输入维度为 d_ff，输出维度仍为 d_model。你的模型由 num_blocks个这样的两层线性网络块组成。此外，无需进行任何激活检查点优化，并且在计算过程中，所有激活值和梯度通信均采用BF16格式；而累积梯度、主权重及优化器状态则应以FP32格式保存。

(a) 在单个设备上以FP32格式存储主模型权重、累积梯度和优化器状态，需要多少内存？如果采用BF16格式存储反向传播所需的内存（这些将使用BF16），又能节省多少内存？这相当于多少块80GB的H100 GPU所具备的总内存容量？

可交付成果：你的计算过程及一句话回答。

(b) 现在假设你的主权重、优化器状态、梯度以及一半的激活值（在（每秒层）被分片到 NFSDP个设备上。请写出每个设备所需内存的表达式。为了使总内存成本低于1个v5p TPU（每个设备95GB）， NFSDP需要取何值？ **可交付成果**：你的计算过程及一句简要回答。

(c) 仅考虑前向传播过程。根据TPU扩展手册中的数据，使用以下通信带宽： $W_{ici} = 2 \cdot 9 \cdot 10^{10}$，以及TPU v5p的浮点运算性能： $C = 4.6 \cdot 10^{14}$。按照扩展手册的记号，设置 $M_X = 2$，$M_Y = 1$（即三维网格），其中 $X = 16$为您的FSDP维度，而 $Y = 4$则为您的TP维度。请问，在这种配置下，模型的计算能力受限于每设备的批量大小是多少？整体的批量大小又是多少？

交付物：你的计算过程及一句话回答。

(d) 在实践中，我们希望整体批大小尽可能小，同时我们始终使用我们的有效计算（换句话说，我们希望始终避免通信瓶颈）。为了在保持高吞吐量的同时降低模型的批大小，我们还能采用哪些其他技巧？

可交付成果： 请用一段话作答。请以参考文献和/或公式支持你的观点。项。

3 优化器状态分片

分布式数据并行训练在概念上简单且通常非常高效，但要求每个进程持有模型参数和优化器状态的不同副本。这种冗余往往会导致显著的内存开销。例如，AdamW优化器为每个参数维护两个浮点数，这意味着其占用的内存是模型权重的两倍。Rajbhandari等人[2020]描述了几种方法，通过将（1）优化器状态、（2）梯度以及（3）参数在各进程间进行划分，并根据需要在不同工作节点之间通信，从而有效降低数据并行训练中的冗余问题。

在本部分作业中，我们将通过实现优化器状态分片的简化版本，来降低每个Rank的内存消耗。具体而言，我们不会为所有参数都保存优化器状态，而是让每个Rank的优化器实例仅处理一部分参数（大约为1 / 世界规模）。当每个Rank的优化器执行一步更新时，它只会更新其分片中对应的那一部分模型参数。随后，各Rank会将其更新后的参数广播给其他Rank，以确保在每一步优化器更新后，模型参数始终保持同步。

问题（优化器状态分片）：15分

实现一个Python类，用于处理优化器状态的分片。该类应封装任意输入的PyTorch optim.Optimizer，并在每次优化器步骤后负责同步更新的参数。我们建议采用以下公共接口：

def __init__(self, params, optimizer_cls: Type[Optimizer], **kwargs: Any): 初始化分片状态优化器。 params 是待优化的参数集合（或参数组，如果用户希望为模型的不同部分使用不同的超参数，例如学习率）；这些参数将在所有进程间进行分片。 optimizer_cls 参数指定了要封装的优化器类型（例如， optim.AdamW）。最后，任何剩余的关键字参数将被转发到 optimizer_cls 的构造函数中。请务必在此方法中调用 torch.optim.Optimizer 超类的构造函数。

def step(self, closure, **kwargs): 调用包装优化器的 step() 方法，并传入相应参数；提供了闭包和关键字参数。更新参数后，与其他进程同步。

def add_param_group(self, param_group: dict[str, Any]): 该方法应添加一个参-将组传递给分片优化器。此方法在超类构造函数构建分片优化器时被调用，也可能在训练过程中被调用（例如，用于逐步解冻模型中的各层）。因此，该方法应负责将模型的参数分配到各个进程 ranks 中。

交付物：实现一个容器类，以支持优化器状态的分片处理。为测试你的分片优化器，请先实现适配器 [adapters.get_sharded_optimizer]。随后，运行 uv run pytest tests/test_sharded_optimizer.py 来执行测试。我们建议多次运行测试（例如5次），以确保测试能够稳定通过。

现在我们已经实现了优化器状态分片，接下来让我们分析它对训练过程中峰值内存使用量以及运行时开销的影响。

问题（优化器状态分片计费）：5分

(a) 编写一个脚本，用于分析在启用与禁用优化器状态分片的情况下训练语言模型时的峰值内存使用情况。采用标准配置（1个节点，2块GPU，XL模型尺寸），

报告模型初始化后、优化器步骤执行前以及优化器步骤执行后的峰值内存使用情况。结果是否符合您的预期？请详细分解每种情况下的内存分配（例如：参数占用多少内存，优化器状态占用多少内存，等等）。

交付物：提供2-3句话的响应，包括峰值内存使用情况的结果，以及模型和优化器各组件之间内存分配的具体细分。

(b) 我们的优化器状态分片实现如何影响训练速度？请测量所需时间
在标准配置（1个节点，2块GPU，XL模型尺寸）下，采用和未采用优化器状态分片时每迭代所消耗的资源。

交付物：用2-3句话描述你的计时方案。

(c) 我们采用的优化器状态分片方法与ZeRO第一阶段（即ZeRO-1）有何不同？
DP Pos 在 Rajbhandari 等人，2020 年?

可交付成果：对任何差异进行2-3句总结，尤其关注与内存和通信量相关的部分。

4 尾声

恭喜你完成作业！我们希望你觉得这项任务有趣且富有收获，并且学到了一些关于如何通过提升单GPU速度和/或利用多GPU来加速语言模型训练的技巧。

参考文献

三道。FlashAttention-2：通过更优的并行化与工作划分实现更快的注意力机制，2023年。网址 https://arxiv.org/abs/2307.08691.

崔道、Daniel Y. 付、斯特法诺·埃尔蒙、阿特里·鲁德拉和克里斯托弗·雷。FlashAttention：一种高效且内存友好的精确注意力机制，兼具I/O感知能力。载于Alice H. Oh、Alekh Agarwal、Danielle Belgrave与Kyunghyun Cho主编的《神经信息处理系统进展》，2022年。网址 https://openreview.net/forum?id=H4DqfPSibmx。

马克西姆·米拉科夫和娜塔莉亚·吉梅尔谢因。2018年用于Softmax的在线归一化计算方法。网址 https: //arxiv.org/abs/1805.02867.

霍勒斯·何。从基本原理出发，让深度学习“冷得发抖”。2022年。网址 https://horace.io/brrr_intro.html.

雅各布·奥斯汀、肖尔托·道格拉斯、罗伊·弗罗斯蒂格、安塞尔姆·列夫斯卡娅、查理·陈、沙拉德·维克拉姆、费德里科·莱布龙、彼得·蔡、维奈·拉马塞什、阿尔伯特·韦布森以及雷纳·波普。如何扩展你的模型。2025年。取自：https://jax-ml.github.io/scaling-book/。

赵浩俊、阮福、穆罕默德·梅库里、莱安德罗·韦拉、托马斯·沃尔夫、努阿曼·塔齐，费尔迪南·莫姆。《超大规模实践指南：在GPU集群上训练大模型》，2025年。

萨米亚姆·拉吉班达里、杰夫·拉斯利、奥拉图尼吉·鲁瓦塞和何宇雄。ZeRO：面向训练万亿参数模型的内存优化技术，2020年。arXiv:1910.02054。